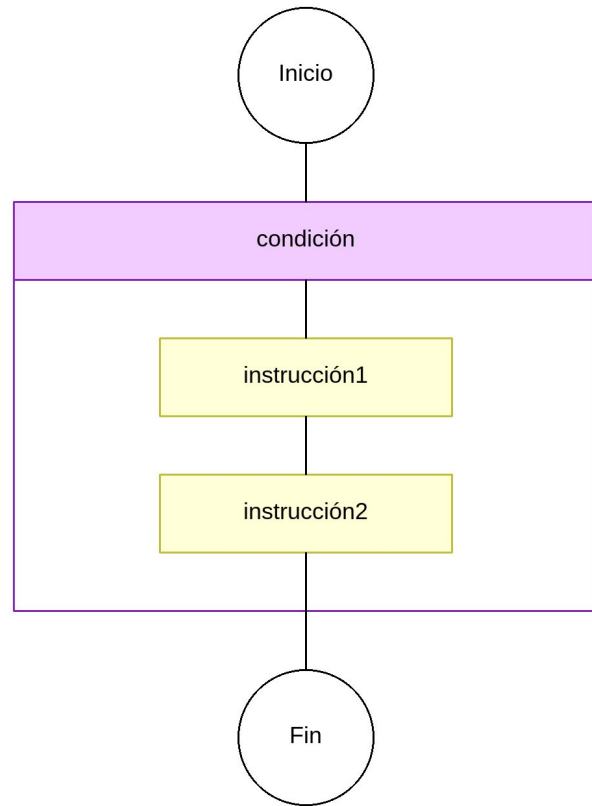


Ciclo inexacto: while

La estructura while

Ciclo inexacto: la cantidad de iteraciones no se conoce de antemano, depende de los datos.



```
while (condición) {  
    instrucción1;  
    instrucción2;  
}
```

Cómo se lee

Mientras la condición sea verdadera, ejecutar el cuerpo y volver a preguntar. Cuando sea falsa, seguir con lo que está después del ciclo.

¿for o while?

Si sabemos cuántas veces repetir, for. Si depende de lo que se ingrese, while.

Caso 1 • Con valor centinela

Un negocio registra los distintos precios de una venta. Se deben ingresar los precios de los productos hasta que se ingresa un precio igual a cero. El cero determina que no hay más productos. Luego, informar por pantalla el **total de la venta** y la **cantidad de productos** vendidos.

- ¿Se sabe de antemano la cantidad de precios que se cargarán?
- ¿Cuándo debe dejarse de ingresar datos?
- ¿Qué representa el cero?
- ¿Es un precio válido para un producto?
- ¿Debería considerarse un producto vendido?

Caso 1 • Prueba de escritorio

El cliente lleva tres productos. El 0 solo avisa que terminó la carga.

INGRESO	precio != 0	total	cantProductos
Inicio		0	0
1200	V	1200	1
850	V	2050	2
3000	V	5050	3
0	F → sale	5050	3

EJECUCIÓN

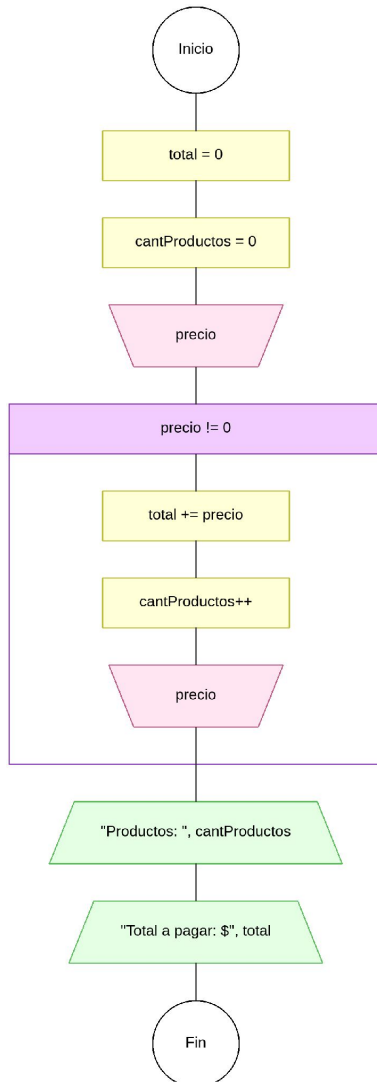
Ingrese el precio (0 para terminar): 1200
Ingrese el precio (0 para terminar): 850
Ingrese el precio (0 para terminar): 3000
Ingrese el precio (0 para terminar): 0
Productos: 3
Total a pagar: \$5050

El centinela no es un dato

El 0 se compara en la condición y el ciclo termina: no se suma al total ni se cuenta como producto.

Caso 1 • Con valor centinela

Caja de supermercado: se ingresan precios hasta que llega un 0. Informar productos y total.



```
float precio, total = 0;
```

```
int cantProductos = 0;
```

```
cin >> precio;
```

```
while (precio != 0) {
```

```
    total += precio;
```

```
    cantProductos++;
```

```
    cin >> precio;
```

```
}
```

```
cout << "Productos: " << cantProductos << endl;
```

```
cout << "Total a pagar: $" << total << endl;
```

1 Primera lectura, afuera

Antes del while ya tiene que haber un precio para comparar.

2 La condición busca el centinela

Mientras no llegue el 0, lo leído es un producto real.

3 Relectura al final del cuerpo

El valor nuevo pasa primero por la condición. Si es 0, se descarta.

Caso 2 · Todos los ingresos se tienen en cuenta

Una persona deposita parte de sus ganancias diarias para poder realizar un viaje. El mismo cuesta \$ 600000. Se desea un programa que permita cargar los depósitos hasta cubrir el importe del viaje. Luego el programa debe mostrar la **cantidad de depósitos** que se realizaron.

- ¿Se sabe de antemano la cantidad de depósitos que se harán?
- ¿Cuándo debe dejarse de ingresar datos?
- ¿Algún ingreso debe ignorarse?
- ¿El depósito que iguala o supera los \$600000 debe considerarse a la cantidad total de depósitos?

Caso 2 · Prueba de escritorio

Tres depósitos. La condición se evalúa antes de cada vuelta.

VUELTA	ahorro < COSTO_VIAJE		deposito	ahorro	cantDep.
Inicio				0	0
1	0 < 600000	V	250000	250000	1
2	250000 < 600000	V	200000	450000	2
3	450000 < 600000	V	180000	630000	3
630000 < 600000 F					

```
EJECUCIÓN

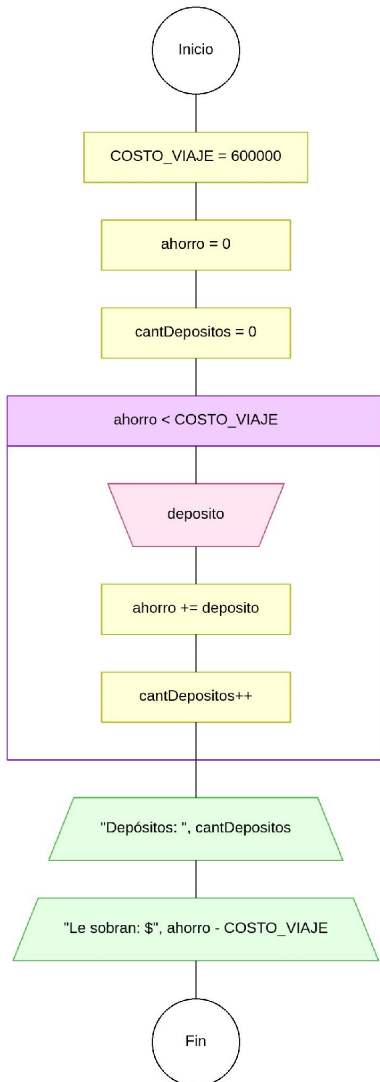
Ingrese el monto depositado: 250000
Ingrese el monto depositado: 200000
Ingrese el monto depositado: 180000
Depósitos: 3
Le sobran: $30000
```

El último dato sí cuenta

Los 180000 de la vuelta 3 se suman y se cuentan. Recién después la condición da falso y el ciclo termina.

Caso 2 · Todos los ingresos se tienen en cuenta

Ahorro para las vacaciones: el viaje cuesta \$600000 y se ingresan depósitos hasta reunirlo.



```
const float COSTO_VIAJE = 600000;

float deposito, ahorro = 0;
int cantDepositos = 0;

while (ahorro < COSTO_VIAJE) {
    cin >> deposito;
    ahorro += deposito;
    cantDepositos++;
}

cout << "Depósitos: " << cantDepositos << endl;
cout << "Le sobran: $" << ahorro - COSTO_VIAJE << endl;
```

1 La condición mira lo acumulado

No hay un valor especial: se sigue mientras falte dinero.

2 Se lee al principio del cuerpo

No hace falta leer antes: ahorro arranca en 0 y la condición ya se puede evaluar.

3 Todo depósito se procesa

Incluso el último, que es justamente el que completa el ahorro.